

HACKEN

RADIX SECURITY ANALYSIS

Intro

This report may contain confidential information about IT systems and the intellectual property of the Customer, as well as information about potential vulnerabilities and methods of their exploitation.

The report can be disclosed publicly after prior consent by another party. Any subsequent publication of this report shall be without mandatory consent.

Name	Radix
Website	https://www.radixdlt.com/
Repository	https://github.com/radixdlt/radixdlt-scrypto
Commit	25474f5391d138e79bfaaf4f5beaa9d3d9757543
Platform	L1
Network	Radix
Languages	Rust
Methods	Automated Code analysis, Manual review, Issues simulation, Fuzz testing
Auditor	b.barwikowski@hacken.io
Approver	l.ciattaglia@hacken.io
Timeline	10.04.2022 - 04.07.2023

Table of contents

- **Summary**
 - Documentation quality
 - Code quality
 - Architecture quality
 - Security score
 - Total score
 - Findings count and definitions
- **Scope of the audit**
 - Protocol audit
 - Protocol tests
- **Issues**
 - Authorization bypass using transaction processor blueprint
 - Inflation bug due to lack of validation
 - Application crash in compose_proof_by_amount
 - Application crash in distribute_fees
 - Application crash in get_auth_zone
 - Application crash in kernel_lock_substate
 - Out of memory crash in log_message and emit_event
 - Application crash in WasmiModule
 - Invalid cache size calculation
- **Tests and code coverage**
- **Disclaimers**
 - Hacken disclaimer
 - Technical disclaimer

Summary

Radix is a layer-1 network designed specifically for DeFi developers, dedicated to eliminating the barriers inhibiting the full potential of DeFi. It provides a unique smart contract environment centered on asset handling, along with a collection of pre-made and reusable DeFi "lego bricks". Radix goal is to cultivate a self-sustaining developer ecosystem with its Developer Royalty System, offering incentives for developers contributing to the network. By implementing its unique Cerberus consensus algorithm and harnessing parallel processing, Radix achieves infinite scalability and seamless interoperability between dApps.

The core focus of the audit is the Radix Engine, the main execution component of the Radix layer-1 network. Built on WASM VM, it offers a specialized environment for running DeFi-centric Scripto applications. The engine uses well-defined finite state machines (FSMs) to control tokens and other assets, guaranteeing secure and predictable DeFi transactions. With the introduction of Radix Engine v2, "smart contract" programmability was brought to the platform through Scripto, fostering asset-oriented, FSM-based financial assets and dApps development. In 2023, the engine's WASM interface received a substantial update, providing a more refined low-level WASM API and improving its Scripto function export signatures. This revision simplified Scripto contract execution, aligning with the network's objective of making DeFi dApp development more efficient and secure.

Scripto is an asset-focused smart contract language employed within the Radix Engine. It empowers developers to effortlessly create and engage with assets on the Radix platform, promoting a safer and more practical DeFi dApp development process. Leveraging Scripto, developers can concentrate on their business logic, entrusting the Radix Engine with asset management and authorization.

Documentation quality

Radix Engine provides a comprehensive array of developer documentation, abundant with example projects and the functionality to generate documentation directly from the source code. This thoroughness significantly aids developers in understanding and utilizing the system efficiently.

However, the area where Radix's documentation falls short is the detailing of the low-level API, particularly with regards to the VM implementation and its functions. While the code is skillfully written and mostly self-explanatory, it lacks in-line comments which can limit the ease of understanding certain functionalities for developers new to the system.

There is a considerable array of native blueprints, node modules, and system modules available for use within the Radix Engine, callable from the VM. Yet, these are not sufficiently documented, creating a potential knowledge gap for developers.

While the average user or developer may not necessarily interact with these components, the lack of documentation can create an impression of inaccessibility or non-use. This, in certain scenarios, could have an impact on both the perceived and actual stability and security of the system. It's recommended to invest more efforts in improving the documentation for these less-explored but potentially crucial areas of the system.

The total Documentation Quality score is **8** out of 10.

Code quality

Radix Engine codebase presents a high standard of craftsmanship, boasting well-written, largely self-explanatory code. The project's structure is well organized, making it relatively straightforward to understand and engage with. It is replete with a variety of tests, including fuzz tests, and incorporates developer-friendly tools for efficient working and debugging. Furthermore, it adheres to good coding practices in Rust.

However, a few issues prevent the codebase from achieving a perfect score. The system's high complexity creates an environment where it is near impossible to accurately test every conceivable scenario. While fuzzing is the most effective method for testing such complex software, this project's implementation of it is insufficient. During the audit, our creation of additional fuzzers allowed for the discovery of numerous issues that would have been difficult to identify manually.

In addition, there is a lack of proper error handling in many places within the code, which led to numerous application crashes during the audit. These issues, although prevalent, can be remedied relatively easily in most cases.

Lastly, the code is riddled with over 200 instances of TODOs and FIXMEs, indicating unfinished tasks and potential issues that need addressing. These unresolved tasks and potential problems should be prioritized and tackled to further improve the code quality.

The total Code Quality score is **7** out of 10.

Architecture quality

The architecture of the Radix Engine is well-conceived and executed, utilizing a contemporary approach for the VM implementation and fostering an impressive ecosystem for users and developers alike. This approach has the potential to greatly enhance the process of "smart contract" development, making it both easier and more secure, even outperforming the Solidity equivalent in user-friendliness.

However, there are a few issues that need to be addressed. Currently, the size of packages ("smart contracts") is rather large, and in many instances, their execution, especially loading can be relatively slow. These are areas that could be improved to enhance the overall performance and efficiency of the system.

While these issues are certainly noteworthy, they are surmountable, and with time and consistent development efforts, they should be rectified. This will further solidify the architecture quality of the Radix Engine.

The architecture quality score is **9** out of 10.

Security score

During the security audit of Radix, eight critical issues were uncovered. The most serious issue allowed for the generation of an unlimited number of new tokens without any need for authorization. The second most serious issue involved the creation of a package ("smart contract") that could reuse a user's authentication when called, allowing it to take all of the user's tokens. Both issues were confirmed and recreated with test cases. The rest of the critical issues identified caused application crash.

As of the commit `98576add49c31fb2609d6d27e8a9a5c14f4e7551` all issues discovered during the audit were correctly fixed or mitigated.

The security score is **10** out of 10.

Total score

Considering all metrics, the total score of the report is **9.4** out of 10.

Findings count and definitions

Severity	Findings	Severity Definition
Critical	8	Vulnerabilities that can lead to a complete breakdown of the blockchain network's security, privacy, integrity, or availability fall under this category. They can disrupt the consensus mechanism, enabling a malicious entity to take control of the majority of nodes or facilitate 51% attacks. In addition, issues that could lead to widespread crashing of nodes, leading to a complete breakdown or significant halt of the network, are also considered critical along with issues that can lead to a massive theft of assets. Immediate attention and mitigation are required.
High	0	High severity vulnerabilities are those that do not immediately risk the complete security or integrity of the network but can cause substantial harm. These are issues that could cause the crashing of several nodes, leading to temporary disruption of the network, or could manipulate the

		consensus mechanism to a certain extent, but not enough to execute a 51% attack. Partial breaches of privacy, unauthorized but limited access to sensitive information, and affecting the reliable execution of smart contracts also fall under this category.
Medium	0	Medium severity vulnerabilities could negatively affect the blockchain protocol but are usually not capable of causing catastrophic damage. These could include vulnerabilities that allow minor breaches of user privacy, can slow down transaction processing, or can lead to relatively small financial losses. It may be possible to exploit these vulnerabilities under specific circumstances, or they may require a high level of access to exploit effectively.
Low	1	Low severity vulnerabilities are minor flaws in the blockchain protocol that might not have a direct impact on security but could cause minor inefficiencies in transaction processing or slight delays in block propagation. They might include vulnerabilities that allow attackers to cause nuisance-level disruptions or are only exploitable under extremely rare and specific conditions. These vulnerabilities should be corrected but do not represent an immediate threat to the system.
Total	9	

Scope of the audit

Protocol audit

VM Engine

- VM implementation review (instructions, state transition, etc)
- Common VM Vulnerabilities review
- Attack scenarios analysis (Gas, race, stack, DoS, state implosion...)

Interpreter

- Interpreter implementation review
- Scrypto integration review

Cryptography and Keys

- Cryptography Libraries
- Keys Generation
- Keystore storage
- Asymmetric (Signing and Verification)

Accounts

- Accounts implementation review
- Security vectors analysis (data availability, nonce,...)

Transactions

- Transaction execution review (execution, fee, receipt)

Storage

- Storage implementation review
- Security aspects analysis (DoS, corruption, state implosion)

Protocol tests

VM Tests

- Environment Setup
- Fuzz tests

Integration tests

- Scrypto

Issues

Authorization bypass using transaction processor blueprint

Is it possible to access and invoke the transaction processor native package from a custom blueprint, thereby bypassing authorization and potentially stealing all user assets when the component `malicious_blueprint` is called.

ID	RDX-105
Scope	Virtual Machine (VM)
Severity	CRITICAL
Vulnerability Type	CWE-285: Improper Authorization
Issue reproduction	issues_reproduction/access_to_transaction_processor
Status	Fixed

Description

The current Virtual Machine (VM) implementation provides the capability to call any package or component from itself through the `call_function` or `call_method` methods. It also facilitates interaction with native components, one of which is the `TransactionProcessorBlueprint`. This component processes and executes transaction manifests.

However, there's a significant vulnerability in this setup. When the `TransactionProcessorBlueprint` is invoked from a blueprint, it can compel a user to execute any manifest. This includes potentially harmful manifests that could withdraw all the user's assets and deposit them into a different account. When such a manifest is processed, it's treated as a user-signed manifest, thereby reusing the user's authorization from the original manifest.

To illustrate this issue, an exploit along with instructions for its use was created. In this scenario, the user who triggers this component will unwittingly transfer 15 Radix tokens to another account, even though their original manifest didn't authorize this action.

It appears from the implementation that the `TransactionProcessorBlueprint` was not intended to be accessible from other packages and components. Besides the problem of bypassing authorization, there are additional issues, such as the system crash that occurs in `manifest_decode(&input.instructions).unwrap()` when incorrect input is provided.

Impact

This vulnerability poses several threats to the protocol:

- **Financial Loss:** The most immediate harm is financial loss. Users' assets can be withdrawn and deposited into a different account without their consent or knowledge, potentially leading to massive financial losses. In a blockchain protocol where transactions are immutable, these losses cannot be reversed once the attack is executed.
- **System Disruption:** The vulnerability allows system crashes when incorrect input is provided, causing significant disruption. This could halt transactions across the entire network, affecting every user and potentially causing a loss of trust in the protocol's reliability.

Recommendation

The following steps are recommended to address this vulnerability:

- 1. Restrict Access:** Limit the accessibility of `TransactionProcessorBlueprint` to prevent it from being invoked by other packages and components. This will mitigate the risk of unauthorized actions being performed without the user's consent.
- 2. Implement Strong Authorization Checks:** Add robust checks before a transaction manifest is executed. Ensure that a new authorization or confirmation is required for each manifest execution, preventing the re-use of user authorization from the original manifest.
- 3. Input Validation:** Strengthen the input validation in `manifest_decode(&input.instructions).unwrap()`. The system should be able to handle incorrect inputs gracefully without leading to crashes. Proper error handling should be implemented to catch and manage exceptions.

Inflation bug due to lack of validation

Numerous blueprints and components in the project inadequately validate the input `NodeId`, causing potential application crashes and at worst, generating XRD tokens illegitimately.

ID	RDX-108
Scope	Virtual Machine (VM)
Severity	CRITICAL
Vulnerability Type	CWE-20: Improper Input Validation
Issue reproduction	issues_reproduction/lack_of_input_validation
Status	Fixed

Description

The current project implementation contains a variety of blueprints and components which accept `Bucket`, `Proof`, or other structures that inherit from the `Own / NodeId` structures as inputs. These structures point to an object's address within the VM, with their serialization and deserialization being identical. The primary concern here is that these inputs are not sufficiently validated to ensure the provided `NodeId` address pertains to the expected blueprints or resources. This lack of validation can lead to serious issues, ranging from application crashes due to deserialization errors to more serious cases of illicit XRD token generation.

In more detail, an input could reference a `Proof` object when a `Bucket` is expected, resulting in a deserialization error and, consequently, an application crash. More alarmingly, it's possible to provide an address for a custom object that behaves like a different object, such as a `Bucket`. This could allow an attacker to create counterfeit buckets capable of generating XRD tokens illegitimately.

An instance of this vulnerability can be found in the `BucketBlueprint::put` function, located at `radix-engine/src/blueprints/resource/buckets.rs`. The relevant code implementation is as follows:

```
pub fn put<Y>(
    receiver: &NodeId,
    input: &IndexedCryptoValue,
    api: &mut Y,
) -> Result<IndexedCryptoValue, RuntimeError>
where
    Y: KernelNodeApi + KernelSubstateApi + ClientApi<RuntimeError>,
{
    let input: BucketPutInput = input.as_typed().map_err(|e| {
        RuntimeError::SystemUpstreamError(SystemUpstreamError::InputDecodeError(e))
    })?;
    // Drop other bucket
    let other_bucket: DroppedBucket = api.kernel_drop_node(input.bucket.0.as_node_id()).into();
    // Check resource address
    let info = BucketInfoSubstate::of(receiver, api)?;
    if info.resource_address != other_bucket.info.resource_address {
        return Err(RuntimeError::ApplicationError(
```

```

        ApplicationError::BucketError(BucketError::MismatchingResource),
    ));
}
// Put
match other_bucket.resource {
    DroppedBucketResource::Fungible(r) => {
        FungibleBucket::put(receiver, r, api)?;
    }
    DroppedBucketResource::NonFungible(r) => {
        NonFungibleBucket::put(receiver, r, api)?;
    }
}
Ok(IndexedScripToValue::from_typed(&()))
}

```

This function is intended to merge two buckets, but a significant issue arises when the address provided for the second bucket does not correspond to an actual bucket, leading to panic due to a serialization error.

A detailed examination of the conversion process from `HeapNode` to `DroppedBucket` reveals the vulnerability:

```

impl Into<DroppedBucket> for HeapNode {
    fn into(mut self) -> DroppedBucket {
        let mut module = self.substates.remove(&SysModuleId::ObjectTuple).unwrap();
        let info: BucketInfoSubstate = module
            .remove(&BucketOffset::Info.into())
            .map(|x| x.as_typed().unwrap())
            .unwrap();
        let resource = match info.resource_type {
            ResourceType::Fungible { .. } => DroppedBucketResource::Fungible(
                module
                    .remove(&BucketOffset::LiquidFungible.into())
                    .map(|x| x.as_typed().unwrap())
                    .unwrap(),
            ),
            ResourceType::NonFungible { .. } => DroppedBucketResource::NonFungible(
                module
                    .remove(&BucketOffset::LiquidNonFungible.into())
                    .map(|x| x.as_typed().unwrap())
                    .unwrap(),
            ),
        };
        DroppedBucket { info, resource }
    }
}

```

As observed, it is feasible to create a custom object that is correctly deserialized to a `BucketInfoSubstate` and a `LiquidFungibleResource`, facilitating the creation of a fraudulent `Bucket`. This fraudulent `Bucket` can then be merged with a genuine `Bucket`, enabling the illicit generation of arbitrary amounts of new tokens.

This flaw is found in 10 different structures that inherit from `Own`, which are used in over 50 places in the codebase. Furthermore, the issue isn't restricted to the native codebase; it also affects third-party blueprints and components. Thus, when a `Bucket` is used, there's no guarantee that the `NodeId` address provided by the user corresponds to the expected blueprint.

Impact

The impact of this vulnerability can be severe due to the potential for illicit token generation and system instability.

- Illicit Token Generation:** The primary risk is the potential creation of fraudulent "buckets" which can generate XRD tokens illegitimately. If exploited, this vulnerability allows the creation of arbitrary numbers of new tokens, potentially leading to massive inflation. Such an event can severely devalue the existing tokens, disrupting the economy of the system and eroding user trust.
- System Instability:** The vulnerability can lead to application crashes due to deserialization errors when an incorrect `NodeId` address is provided. If attackers repeatedly exploit this flaw, it can lead to repeated system crashes, causing denial-of-service conditions and negatively impacting system availability.

3. **Third-Party Risk:** The vulnerability extends to third-party blueprints and components, widening its potential impact. This means the software relying on these components can also be exposed to the same risks of crashes and fraudulent token creation.
4. **Data Integrity:** While the vulnerability primarily affects token generation and system stability, it also opens the potential for data integrity issues. If custom objects can masquerade as other objects like a `Bucket`, there might be other, as-yet-unidentified ways in which the data could be manipulated.

In summary, the vulnerability presents a multi-faceted threat with potential impacts on token economy, system stability, third-party components, and data integrity.

Recommendation

To mitigate this vulnerability, a robust validation mechanism for all `NodeId` addresses passed to native blueprints or structures inheriting from `Own / NodeId` is needed. The system should verify whether the address matches the correct object, even for third-party blueprints. An in-depth review of the entire codebase is recommended, focusing on sections that process user inputs, including low-level functions. Any areas lacking sufficient input validation should be addressed promptly by introducing necessary validation measures. It is also advised to develop and employ fuzzing tests to cover these validation functions, as they can help expose potential flaws in the validation process.

Application crash in `compose_proof_by_amount`

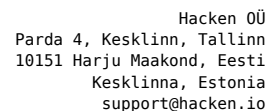
The function `compose_proof_by_amount` located in `radix-engine/src/blueprints/resource/auth_zone_composition.rs` will trigger a panic error if the specified amount is excessively large.

ID	RDX-100
Scope	Radix engine
Severity	CRITICAL
Vulnerability Type	CWE-248: Uncaught Exception (panic)
Status	Fixed

Description

The function `compose_proof_by_amount` within the Rust code:

```
pub fn compose_proof_by_amount<Y: KernelSubstateApi + ClientApi<RuntimeError>>(<br>    proofs: &[Proof],<br>    resource_address: ResourceAddress,<br>    amount: Option<Decimal>,<br>    api: &mut Y,<br>) -> Result<ComposedProof, RuntimeError> {<br>    let resource_type = ResourceManager(resource_address).resource_type(api)?;<br>    if let Some(amount) = amount {<br>        if !resource_type.check_amount(amount) {<br>            return Err(RuntimeError::ApplicationError(<br>                ApplicationError::AuthZoneError(AuthZoneError::ComposeProofError(<br>                    ComposeProofError::InvalidAmount,<br>                )),<br>            ));<br>        }<br>    }<br>    match resource_type {<br>        ...<br>        ResourceType::NonFungible { .. } => compose_non_fungible_proof(<br>            proofs,<br>            resource_address,<br>            match amount {<br>                Some(amount) => NonFungiblesSpecification::Some(</pre>
```



contains a segment of code that converts a `Decimal` amount to `use` when `resource_type` equals `ResourceType::NonFungible`:

The issue arises when the `Decimal` amount exceeds the `usize` limit, which is `2^64 - 1`. If the amount is larger, the `parse()` function will panic, resulting in a `ParseIntError { kind: PosOverflow }` error.

This can be reproduced using the following manifest (ensure the `Address` is replaced with a valid account):

Impact

This vulnerability has significant impacts on the operation and reliability of the protocol, given that it can be intentionally exploited to cause a system crash, disrupting the normal operation.

- **System Stability:** A panic disrupt the normal operation of the system, causing a halt in transaction processing and potentially leading to significant disruption in the blockchain network.
- **Denial of Service (DoS) Attacks:** Attackers could intentionally exploit this vulnerability to execute a Denial of Service (DoS) attack. By crafting and sending transactions that include amounts exceeding the use limit, they could repeatedly cause the system to panic and stop executing, preventing other legitimate transactions from being processed.

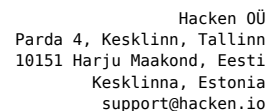
Recommendation

Instead of using `expect` there should be correct error handling. To mitigate this issue, a robust error handling mechanism should be incorporated, replacing the current `expect` usage. This will ensure that when the `Decimal` amount exceeds the `usize` limit, the function doesn't panic but handles the error gracefully, perhaps by returning an `Err` variant containing a specific error message.

Application crash in distribute_fees

The `distribute_fees` function, located in `radix-engine/src/transaction/transaction_executor.rs`, will cause a panic event when a transaction attempts to create an XRD bucket without sufficient balance.

ID	RDX-101
Scope	Radix engine



Description

```
// Refund overpayment
let handle = track
    .acquire_lock(
        &vault_id,
        SysModuleId::ObjectTuple.into(),
        &FungibleVaultOffset::LiquidFungible.into(),
        LockFlags::MUTABLE,
    )
    .unwrap();
```

This vulnerability can be replicated with the following manifest (ensure the `Address` is replaced with a valid account):

[illegible]

Impact

- **System Stability:** A panic disrupts the normal operation of the system, causing a halt in transaction processing and potentially leading to significant disruption in the blockchain network.
- **Denial of Service (DoS) Attacks:** Attackers could intentionally exploit this vulnerability to execute a Denial of Service (DoS) attack. By crafting and sending transactions that include amounts exceeding the size limit, they could repeatedly cause the system to panic and stop executing, preventing other legitimate transactions from being processed.

Recommendation

To rectify this issue, it is crucial that the lock for the XRD vault is appropriately released when the creation of the `bucket` fails. However, this adjustment should be implemented with caution as it could inadvertently trigger other issues within the system. This underscores the need for comprehensive testing to ensure this fix does not introduce new vulnerabilities or problems.

Application crash in get_auth_zone

The `api.get_auth_zone().unwrap()` function is invoked at various points in the code. However, it may cause a system crash, or panic, when it encounters an error due to insufficient fee reserves and attempts to perform an 'unwrap' operation on the error result.

ID	RDX-102
Scope	Radix engine
Severity	CRITICAL
Vulnerability Type	CWE-248: Uncaught Exception (panic)
Issue reproduction	issues_reproduction/get_auth_zone_unwrap_panic
Status	Fixed

Description

The `get_auth_zone()` function, in its current implementation, deducts a `FIXED_LOW_FEE` cost unit each time it's invoked:

```
fn get_auth_zone(&mut self) -> Result<NodeId, RuntimeError> {  
    self.consume_cost_units(FIXED_LOW_FEE, ClientCostingReason::RunSystem)?;  
    let auth_zone_id = self.api.kernel_get_callback().modules.auth.last_auth_zone();  
    Ok(auth_zone_id.into())  
}
```

Under circumstances where fee reserves are exhausted, this function may yield a `FeeReserveError` as a result. By crafting a specific type of manifest, one could deplete the fee reserve, causing this function to return an error.

The issue is compounded by the fact that the `get_auth_zone().unwrap()` is utilized in 13 different locations. This practice can trigger a panic and subsequent application crash when the function encounters an error. An instance of such usage can be found in the `sys_drain` function in `native-sdk/src/resources/auth_zone.rs`:

```
pub fn sys_drain<Y, E: Debug + ScryptoCategorize + ScryptoDecode>(  
    api: &mut Y,  
) -> Result<Vec<Proof>, E>  
where  
    Y: ClientApi<E>,  
{  
    let auth_zone = api.get_auth_zone().unwrap();  
    let rtn = api.call_method(  
        &auth_zone,  
        AUTH_ZONE_DRAIN_IDENT,  
        scrypto_encode(&AuthZoneDrainInput {}).unwrap(),  
    )?;  
    Ok(scrypto_decode(&rtn).unwrap())  
}
```

Impact

This vulnerability has significant impacts on the operation and reliability of the protocol, given that it can be intentionally exploited to cause a system crash, disrupting the normal operation.

- **System Stability:** A panic disrupt the normal operation of the system, causing a halt in transaction processing and potentially leading to significant disruption in the blockchain network.
- **Denial of Service (DoS) Attacks:** Attackers could intentionally exploit this vulnerability to execute a Denial of Service (DoS) attack. By crafting and sending transactions that include amounts exceeding the usage limit, they could repeatedly cause the system to panic.

and stop executing, preventing other legitimate transactions from being processed.

Recommendation

To prevent potential crashes, it is advisable to eliminate the usage of `unwrap` when invoking the `get_auth_zone()` function. Instead, proper error handling mechanisms should be incorporated. These mechanisms would gracefully handle scenarios where the function might return an error, thus enhancing the overall resilience of the application. It is vital to review each occurrence of `unwrap()` with the `get_auth_zone()` function to ensure the integrity of the system and avoid abrupt crashes.

Application crash in `kernel_lock_substate`

The function `kernel_lock_substate` has a "hack" which can cause application crash.

ID	RDX-106
Scope	Virtual Machine (VM)
Severity	CRITICAL
Vulnerability Type	CWE-789: Uncontrolled Memory Allocation
Status	Fixed

Description

The `kernel_lock_substate` in `kernel.rs` has the following implementation:

```
fn kernel_lock_substate(
    &mut self,
    node_id: &NodeId,
    module_id: SysModuleId,
    substate_key: &SubstateKey,
    flags: LockFlags,
) -> Result<LockHandle, RuntimeError> {
    M::before_lock_substate(&node_id, &module_id, substate_key, &flags, self)?;
    let maybe_lock_handle = self.current_frame.acquire_lock(
        &mut self.heap,
        self.store,
        node_id,
        module_id,
        substate_key,
        flags,
    );
    let lock_handle = match &maybe_lock_handle {
        Ok(lock_handle) => *lock_handle,
        Err(LockSubstateError::TrackError(track_err)) => {
            ...
        }
    }
    Err(err) => {
        match &err {
            // TODO: This is a hack to allow for package imports to be visible
            // TODO: Remove this once we are able to get this information through the Blueprint ABI
            LockSubstateError::NodeNotInCallFrame(node_id)
                if node_id.is_global_package() =>
            {
                let module_id = SysModuleId::ObjectTuple;
                ...
                self.current_frame.add_ref(*node_id, RefType::Normal);
                self.current_frame
                    .acquire_lock(
                        &mut self.heap,
                        self.store,
```

```

        &node_id,
        module_id,
        substate_key,
        flags,
    )
    .map_err(CallFrameError::LockSubstateError)
    .map_err(KernelError::CallFrameError)?
}
=> {
    return Err(RuntimeError::KernelError(KernelError::CallFrameError(
        CallFrameError::LockSubstateError(err.clone()),
    )))
}
}
}
};
// TODO: pass the right size
M::after_lock_substate(lock_handle, 0, self)?;
Ok(lock_handle)
}

```

When application tries to lock substate of the global package and fails, there's a hack which will "allow for package imports to be visible". The problem with this hack is, that it changes `module_id` to `SysModuleId::ObjectTuple` which may return different data than expected.

In the sources, there are 34 places in the code which use `substate.as_typed().unwrap()` or similar syntax which will cause panic when this hack is used. The issue was detected and reproduced by the WASM fuzzer in the following places:

```

panicked at 'called `Result::unwrap()` on an `Err` value: UnexpectedSize { expected: 2, actual: 3 }',
/src/radix/radix-engine/src/system/node_modules/access_rules/package.rs:457:62
panicked at 'called `Result::unwrap()` on an `Err` value: UnexpectedSize { expected: 3, actual: 2 }',
/src/radix/radix-engine/src/kernel/heap.rs:143:35
panicked at 'called `Result::unwrap()` on an `Err` value: UnexpectedValueKind { expected: 34, actual: 33 }',
/src/radix-engine/src/system/node_modules/type_info/package.rs:29:83

```

Impact

This vulnerability has significant impacts on the operation and reliability of the protocol, given that it can be intentionally exploited to cause a system crash, disrupting the normal operation.

- **System Stability:** A panic disrupt the normal operation of the system, causing a halt in transaction processing and potentially leading to significant disruption in the blockchain network.
- **Denial of Service (DoS) Attacks:** Attackers could intentionally exploit this vulnerability to execute a Denial of Service (DoS) attack. By crafting and sending transactions that include amounts exceeding the usage limit, they could repeatedly cause the system to panic and stop executing, preventing other legitimate transactions from being processed.

Recommendation

Instead of assuming the `module_id` is `SysModuleId::ObjectTuple` during the hack process, it would be safer to verify it explicitly. This can prevent data discrepancy errors when interacting with other parts of the application. Therefore, when executing this workaround, make sure to check the `module_id` matches `SysModuleId::ObjectTuple` and flags adhere to `LockFlags::read_only`. This will ensure the correct context for the operation, helping avoid any unexpected panic events due to inconsistent data. Moreover, consider revisiting this "hack" altogether to find a more reliable and stable solution that eliminates the need for these kinds of overrides in your code. As a final safeguard, throughout your codebase, consider implementing comprehensive error handling mechanisms to gracefully handle `unwrap()` failures instead of triggering system panics.

Out of memory crash in log_message and emit_event

The `log_message` and `emit_event` functions, if not properly controlled, have the potential to consume approximately 128 GB of memory, a situation which could lead to an application crash.

ID	RDX-107
Scope	VM
Severity	CRITICAL
Vulnerability Type	CWE-789: Uncontrolled Memory Allocation
Status	Fixed

Description

The `log_message` and `emit_event` functions in the Virtual Machine (VM) have been identified to possess an uncontrolled memory allocation vulnerability. This vulnerability, originating from the lack of a maximum usage limit, can trigger a catastrophic out-of-memory (OOM) crash.

The `log_message` function, as detailed in its implementation, has an unregulated behavior of logging messages without an upper bound:

```
fn log_message(&mut self, level: Level, message: String) -> Result<(), RuntimeError> {
    self.consume_cost_units(FIXED_LOW_FEE, ClientCostingReason::RunSystem?);
    self.api
        .kernel_get_callback()
        .modules
        .logger
        .add_log(level, message);
    Ok(())
}
pub fn add_log(&mut self, level: Level, message: String) {
    self.0.push((level, message))
}
```

The function consumes `FIXED_LOW_FEE` units and appends the message to a `Vec<(Level, String)>`. The issue arises due to the fact that a message can be up to 4 MB in size and the function could be invoked countless times, subject to the client's financial capability. Given the current system's fee model, it is plausible to invoke `log_message` approximately 30,000 times in a single transaction when the maximum fee of 10 XRD is utilized. Consequently, a call to `log_message` for about 30,000 times, each with a 4 MB message, can cause the system to exhaust around 128 GB of memory, which may lead to an OOM crash in nodes with insufficient memory.

The vulnerability can be exploited using the following WAT code:

```
(module
  (import "env" "log_message" (func $log_message (param i32 i32 i32 i32)))
  (data (i32.const 0) "\5C\22\01\00TEXT")
  (func $test (param $0 i64) (result i64)
    (loop
      (call $log_message
        (i32.const 0)
        (i32.const 4)
        (i32.const 4)
        (i32.const 4190208)
      )
      (br 0)
    )
  )
  (memory $0 64)
  (export "memory" (memory $0))
  (export "test" (func $test))
)
```

In this exploit, the string `TEXT` should be substituted with a string of 4,190,208 characters.

The `emit_event` function is vulnerable to similar exploits, although it necessitates more complex encoding to be effectively utilized.

Impact

The immediate impact of an OOM crash would be network disruption. Affected nodes could become unresponsive or completely unavailable, hindering the processing of transactions and potentially leading to significant delays in the addition of new blocks to the blockchain.

This disruption could also lead to a temporary centralization of the network, as nodes with less memory capacity might be more prone to crash, leaving only the more robust ones functioning. This scenario contradicts the decentralization principle fundamental to blockchain systems.

In addition to operational issues, an OOM crash might also expose the system to further security threats. As the system struggles to recover, it might become more vulnerable to additional types of attacks, such as double-spending or 51% attacks.

Moreover, frequent crashes could erode trust in the blockchain system. Confidence in the system's reliability and robustness is critical for users and stakeholders, and any compromise to system performance or security could lead to diminished user engagement and potential loss of value in the associated cryptocurrency.

Recommendation

To address this vulnerability, it is recommended to implement a memory usage limit for the `log_message` and `emit_event` functions. Consider introducing a rate limit on the number of calls allowed to these functions within a single transaction. Also, an upper bound on the size of individual log messages and events could prevent disproportionate memory usage. This would ensure that the memory usage remains within a manageable limit, preventing potential OOM crashes and maintaining system stability.

Application crash in WasmiModule

There's a specific WASM code which will cause panic when `WasmiModule::new` is called.

ID	RDX-103
Scope	Virtual Machine (VM)
Severity	CRITICAL
Vulnerability Type	CWE-248: Uncaught Exception (panic)
Issue reproduction	issues_reproduction/wasmi_module_panic
Status	Fixed

Description

A vulnerability was detected by a fuzzer in the `WasmiModule::new` function within `wasmi.rs`, resulting in an application crash.

The function's implementation is as follows:

```
pub fn new(code: &[u8]) -> Result<Self, PrepareError> {
    let engine = Engine::default();
    let module = Module::new(&engine, code).expect("WASM undecodable, prepare step missed?");
    let mut store = Store::new(&engine, WasmiInstanceEnv::new());
    ...
}
```

The vulnerability lies in the error handling mechanism of the function. It is possible to create a specific WASM code, which although it passes all validations, triggers an error of type `Module(Translation(TranslationError { inner: Validate(BinaryReaderError { inner: BinaryReaderErrorInner { message: "too many locals: locals exceed maximum", offset: 76, needed_hint: None } })` when calling `Module::new(&engine, code)`.

This error is currently unhandled and causes an application panic with the message "WASM undecodable, prepare step missed?", which results in an application crash.

The WASM code that triggers this crash is a non-decodable binary code. This code, along with its manifest and package schema, is attached to the report.

It's worth noting that this issue might also exist in Wasmer's implementation of the `Module::new` function, though this has not been verified.

Impact

This vulnerability has significant impacts on the operation and reliability of the protocol, given that it can be intentionally exploited to cause a system crash, disrupting the normal operation.

- **System Stability:** A panic disrupt the normal operation of the system, causing a halt in transaction processing and potentially leading to significant disruption in the blockchain network.
- **Denial of Service (DoS) Attacks:** Attackers could intentionally exploit this vulnerability to execute a Denial of Service (DoS) attack. By crafting and sending transactions that include amounts exceeding the usize limit, they could repeatedly cause the system to panic and stop executing, preventing other legitimate transactions from being processed.

Recommendation

The use of `expect("WASM undecodable, prepare step missed?")` when invoking `Module::new(&engine, code)` lacks proper error handling, leading to unexpected application crashes. It is recommended to implement a robust error handling mechanism in its place.

In the case where the module is undecodable, the function should gracefully handle the error by returning a `PrepareError` or a similar error indicating the failure in WASM decoding, rather than causing a panic.

Invalid cache size calculation

Cache implementation in Wasmi and Wasmer calculates cache size incorrectly.

ID	RDX-104
Scope	Virtual Machine (VM)
Severity	LOW
Vulnerability Type	CWE-789: Uncontrolled Memory Allocation
Status	Fixed

Description

The current cache implementation in `wasmi.rs` (and `wasmer.rs`), has the following implementation:

The cache strategy in the current implementation of `wasmi.rs` (and `wasmer.rs`) determines the cache size primarily based on the size of the cached code (`code_size_bytes`), without considering the memory overhead from `Store<FakeHostState>`, which hosts `WasmRuntime`:

```
pub struct WasmiModule {
    template_store: Store<FakeHostState>,
    template_instance: Instance,
    #[allow(dead_code)]
    code_size_bytes: usize,
}

pub struct WasmiEngine {
    modules_cache: moka::sync::Cache<MeteredCodeKey, Arc<WasmiModule>>,
}

let modules_cache = moka::sync::Cache::builder()
    .weigher(|_key: &MeteredCodeKey, value: &Arc<WasmiModule>| -> u32 {
        // Approximate the module entry size by the code size
        value.code_size_bytes.try_into().unwrap_or(u32::MAX)
    })
    .max_capacity(options.max_cache_size_bytes as u64)
    .build();
```

This approach is problematic because each instance of `WasmiModule` carries an overhead of around 50KB-200KB due to `Store<FakeHostState>`. Although the cache limit is set to 200 MB by default, this overhead led to excessive memory consumption during fuzz testing. With thousands of small-sized modules in the cache, each contributing an additional overhead, the memory usage surpassed 2 GB, leading to an application crash.

Given the high cost of creating packages, the likelihood of reproducing this issue in a real-world scenario is low, making the severity of this issue comparatively minimal.

Recommendation

To ensure accurate cache sizing, the calculation should account for the overhead introduced by `Store<FakeHostState>`. This adjustment will provide a true reflection of memory usage, thereby preventing potential memory exhaustion and application crashes during intensive operations like fuzz testing.

Tests and code coverage

The project includes a comprehensive set of 1800 tests, out of which 7 are currently marked to be ignored. The `radix-engine-tests` represent the most significant and complex part of this testing suite, containing 473 individual tests. However, 6 of these tests within the `radix-engine-tests` are presently being overlooked.

In addition to the test suite, the project also incorporates a fuzz target dedicated to transaction execution. This target uses a seed corpus, which is generated from transactions featured in the test suite itself.

The project has the following code coverage:

Module	Regions (Covered/Total/%)	Functions (Covered/Total/%)	Lines (Covered/Total/%)
native-sdk	309 / 412 / 75.00%	67 / 82 / 81.71%	1014 / 1130 / 89.73%
radix-engine	12798 / 21355 / 59.93%	2993 / 5072 / 59.01%	29020 / 34419 / 84.31%
radix-engine-common	2570 / 3428 / 74.97%	1022 / 1383 / 73.90%	5129 / 6126 / 83.73%
radix-engine-constants	1 / 1 / 100.00%	1 / 1 / 100.00%	1 / 1 / 100.00%
radix-engine-derive	89 / 107 / 83.18%	57 / 68 / 83.82%	632 / 652 / 96.93%
radix-engine-interface	825 / 3246 / 25.42%	445 / 1260 / 35.32%	1204 / 2269 / 53.06%
radix-engine-stores	629 / 1012 / 62.15%	235 / 394 / 59.64%	1810 / 2221 / 81.49%
radix-engine-utils	3 / 3 / 100.00%	3 / 3 / 100.00%	5 / 5 / 100.00%
sbor	2368 / 3789 / 62.50%	586 / 809 / 72.44%	6106 / 7087 / 86.16%
sbor-derive	502 / 604 / 83.11%	119 / 126 / 94.44%	1774 / 1840 / 96.41%
sbor-derive-common	493 / 588 / 83.84%	114 / 115 / 99.13%	1753 / 1809 / 96.90%
scrypto	435 / 1066 / 40.81%	141 / 488 / 28.89%	2080 / 4781 / 43.51%
scrypto-derive	228 / 317 / 71.92%	31 / 36 / 86.11%	807 / 910 / 88.68%
scrypto-schema	22 / 85 / 25.88%	15 / 30 / 50.00%	43 / 71 / 60.56%
scrypto-unit	180 / 204 / 88.24%	93 / 97 / 95.88%	1224 / 1276 / 95

The most important part of the project, `radix-engine` has the following code coverage:

Module	Regions (Covered/Total/%)	Functions (Covered/Total/%)	Lines (Covered/Total/%)
blueprints/access_controller	329 / 519 / 63.39%	55 / 86 / 63.95%	995 / 1076 / 92.47%
blueprints/account	354 / 566 / 62.54%	34 / 62 / 54.84%	724 / 860 / 84.19%
blueprints/clock	74 / 109 / 67.89%	8 / 15 / 53.33%	218 / 239 / 91.21%
blueprints/epoch_manager	451 / 716 / 62.99%	29 / 65 / 44.62%	1105 / 1203 / 91.85%
blueprints/epoch_manager/events	28 / 56 / 50.00%	19 / 24 / 79.17%	19 / 24 / 79.17%

blueprints/identity	81 / 133 / 60.90%	9 / 15 / 60.00%	167 / 219 / 76.26%
blueprints/package	159 / 254 / 62.60%	17 / 33 / 51.52%	453 / 505 / 89.70%
blueprints/resource	1575 / 2433 / 64.73%	116 / 244 / 47.54%	3622 / 4156 / 87.15%
blueprints/resource/events	38 / 77 / 49.35%	26 / 28 / 92.86%	26 / 28 / 92.86%
blueprints/resource/fungible	312 / 454 / 68.72%	33 / 45 / 73.33%	653 / 713 / 91.58%
blueprints/resource/non_fungible	531 / 784 / 67.73%	47 / 62 / 75.81%	1132 / 1270 / 89.13%
blueprints/transaction_processor	289 / 436 / 66.28%	21 / 31 / 67.74%	577 / 666 / 86.64%
blueprints/util	54 / 69 / 78.26%	8 / 8 / 100.00%	105 / 108 / 97.22%
kernel	523 / 724 / 72.24%	86 / 129 / 66.67%	1085 / 1202 / 90.27%
system	919 / 1363 / 67.42%	116 / 151 / 76.82%	2370 / 2651 / 89.40%
system/node_modules/access_rules	222 / 342 / 64.91%	23 / 46 / 50.00%	619 / 679 / 91.16%
system/node_modules/metadata	109 / 177 / 61.58%	12 / 22 / 54.55%	257 / 296 / 86.82%
system/node_modules/royalty	54 / 77 / 70.13%	6 / 9 / 66.67%	165 / 180 / 91.67%
system/node_modules/type_info	11 / 25 / 44.00%	2 / 5 / 40.00%	18 / 22 / 81.82%
system/system_modules/auth	374 / 585 / 63.93%	46 / 80 / 57.50%	794 / 907 / 87.54%
system/system_modules/costing	296 / 462 / 64.07%	89 / 117 / 76.07%	706 / 771 / 91.57%
system/system_modules/events	7 / 24 / 29.17%	5 / 9 / 55.56%	9 / 14 / 64.29%
system/system_modules/execution_trace	201 / 334 / 60.18%	49 / 79 / 62.03%	496 / 541 / 91.68%
system/system_modules/kernel_trace	11 / 14 / 78.57%	11 / 14 / 78.57%	123 / 132 / 93.18%
system/system_modules/logger	3 / 5 / 60.00%	3 / 5 / 60.00%	9 / 11 / 81.82%
system/system_modules/node_move	55 / 75 / 73.33%	6 / 10 / 60.00%	78 / 84 / 92.86%
system/system_modules/transaction_limits	43 / 68 / 63.24%	13 / 15 / 86.67%	143 / 156 / 91.67%
system/system_modules/transaction_runtime	7 / 9 / 77.78%	4 / 6 / 66.67%	54 / 56 / 96.43%
system/system_modules/virtualization	15 / 24 / 62.50%	2 / 4 / 50.00%	43 / 55 / 78.18%
track	76 / 97 / 78.35%	22 / 30 / 73.33%	255 / 277 / 92.06%
transaction	400 / 629 / 63.59%	77 / 127 / 60.63%	1073 / 1295 / 82.86%
utils	16 / 21 / 76.19%	2 / 3 / 66.67%	29 / 30 / 96.67%
vm	217 / 289 / 75.09%	34 / 69 / 49.28%	369 / 513 / 71.93%
vm/wasm	777 / 1189 / 65.35%	165 / 219 / 75.34%	1686 / 2016 / 83.63%

Disclaimers

Hacken disclaimer

The code base provided for audit has been analyzed according to the latest industry code quality, software processes and cybersecurity practices at the date of this report, with discovered security vulnerabilities and issues the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functional specifications). The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code (branch/tag/commit hash) submitted to and reviewed, so it may not be relevant to any other branch. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits, public bug bounty program and CI/CD process to ensure security and code quality. English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

Technical disclaimer

Protocol Level Systems are deployed and executed on hardware and software underlying platforms and platform dependencies (Operating System, System Libraries, Runtime Virtual Machines, linked libraries, etc.). The platform, programming languages, and other software related to the Protocol Level System may have vulnerabilities that can lead to security issues and exploits. Thus, Consultant cannot guarantee the explicit security of the Protocol system in full execution environment stack (hardware, OS, libraries, etc.)